



Shor Algorithm

The Problem

- Shor's algorithm is one of the most famous quantum algorithms, created by mathematician Peter Shor in 1994.
- It showed, for the first time, that a quantum computer could solve a problem much faster than any known classical method.
- The problem it solves is called integer factorization, breaking a large number N into the prime numbers that multiply to make it:
- $$N = p \times q$$
- This is important because the security of many encryption systems, such as RSA, is based on the fact that factoring large numbers is slow on classical computers

How it Works

- Choose a random number that is smaller than N
- Then, using quantum computing, find the period of the function $f(x) = a^x \bmod N$

$$a^r \equiv 1 \pmod{N}$$

- The period is the smallest number r such that: $a^r \equiv 1 \pmod{N}$
- Once r is known, it can be used to find the factors of N efficiently.
- Finding the period r is where quantum computing excels.
- A classical computer would need to test many values of x , but a quantum computer can find r quickly using a process called the Quantum Fourier Transform (QFT).
- Once the period is known, the remaining steps are simple math done on a normal computer to find the factors

Shor's Approach

Shor's algorithm is a quantum algorithm designed to factor large composite integers efficiently, which is exponentially faster than the best known classical algorithms for large numbers.

It leverages quantum superposition, entanglement, and the quantum Fourier transform (QFT) to solve the problem.

The core idea is to reduce factorization to finding the period (or order) of a modular exponentiation function, which is hard classically but efficient quantumly.

Overview

Given a large composite number N that is a product of two large primes:

- Pick a random base (a): Choose a random integer a ($1 < a < N$) that is coprime to N (i.e., $\gcd(a, N) = 1$).
- If not coprime, you've already found a factor via $\gcd$.
- Define the periodic function: Consider the function $f(x) = a^x \bmod N$.
- This function is periodic with some period r , where r is the smallest positive integer such that $a^r \equiv 1 \bmod N$ (this r is called the order of a modulo N).
- Find the period r : This is the key step, done quantumly.
- Once you have r , check if it's even. If not, pick a new a and repeat.

Shor Algorithm

- Pick a number a , which is a co-prime with N . (Classical computer);
- Need to find the smallest r such that $a^r \equiv 1 \pmod{N}$.
- If r is even: Go to the next step. If r is odd go back to pick another a .
- If r is even then we found the non-trivial square root of N ;
- Define $x = \text{pow}(a, r/2)$. (on a Classical computer);
- If $x+1 \equiv 0 \pmod{N}$; then one of p, q is contained in $\text{gcd}(x+1, N)$ or $\text{gcd}(x-1, N)$. (Classical computer);
- If these fail (from step 3); we start everything by considering different a .

The Role of Quantum Computing

- The main point of Shor's algorithm is to find the non-trivial square root of the number that we would like to factorize i.e. N .
- As N gets larger, the calculations become exponentially heavy.
- Shor's great idea was to propose a quantum algorithm to find the order (period) r for which we get the non-trivial square root and then *the whole problem of factoring boils down to order finding*.
- The algorithm partially runs on a quantum computer to find the order r , sandwiched between classical pre and post-processing.
- The next question should be: How we can use quantum computing for order finding?

Simple Example for finding the period (order) r

- **Step 1:** $N = 15$.
- **Step 2:** Pick the first coprime $a = 2$ (randomly; $\gcd(2, 15) = 1$, so it's coprime).
- **Step 3:** Compute $f(x) = 2^x \bmod 15$ for $x = 0, 1, 2, \dots$:
 - $2^0 = 1 \bmod 15$
 - $2^1 = 2 \bmod 15$
 - $2^2 = 4 \bmod 15$
 - $2^3 = 8 \bmod 15$
 - $2^4 = 16 \equiv 1 \bmod 15$ (back to 1, so period $r = 4$).
- **Step 4:** $r = 4$ (found via period search; quantumly, this is fast).
- **Step 5:** r is even, so compute $a^{\{r/2\}} = 2^2 = 4$.
 - $\gcd(4 - 1, 15) = \gcd(3, 15) = 3$
 - $\gcd(4 + 1, 15) = \gcd(5, 15) = 5$
 - **Result:** Factors are 3 and 5.

How the Quantum Fourier Transform Speeds the Period Computation

- The speedup in Shor's algorithm comes from using the **Quantum Fourier Transform (QFT)**, a quantum analog of the classical Discrete Fourier Transform (DFT).
- The QFT efficiently extracts periodicities (frequencies) from a superposition of states, allowing period finding in polynomial time instead of exponential.

Why Fourier Transform Helps

- The period r manifests as a repeating pattern in the function $f(x) = a^x \bmod N$.
- In quantum computing, we create a superposition over many x values: $|\psi\rangle = (1/\sqrt{Q}) \sum |x\rangle |a^x \bmod N\rangle$, where Q is a power of 2 larger than N^2 (to ensure accuracy).
- Measuring the second register ($|a^x \bmod N\rangle$) collapses it to some value s , leaving the first register in a superposition of x where $a^x \equiv s \bmod N$. These x are arithmetic progressions with difference r : $x_0, x_0+r, x_0+2r, \dots$, creating a periodic signal with period r .
- Applying QFT to this periodic superposition transforms it into peaks at frequencies that are multiples of Q/r (approximately).
- Measuring after QFT gives a value y close to $k^*(Q/r)$ for integer k , from which we can estimate $1/r$ (hence r) using continued fractions approximation.

How QFT Speeds the Period Computation

- The QFT is like DFT: For a periodic signal with period r , DFT outputs spikes at frequency $1/r$ (and harmonics). Classically, DFT on M points takes $O(M \log M)$ time via FFT, but to resolve r up to N , M needs to be $\sim N^2$, and evaluating $a^x \bmod N$ for all points is $O(N^2 \log N)$ time—still too slow for large N .
- Quantumly:
- Superposition evaluates $a^x \bmod N$ for all x "in parallel" via quantum parallelism.
- QFT is implemented with $O((\log Q)^2) = O((\log N)^2)$ gates, exponentially faster.
- Overall: Shor's runs in $O((\log N)^3)$ time on a quantum computer.

Simple Illustration with Fourier Transform

- Suppose a signal like $[1,0,0,1,0,0,1,0,0,\dots]$ (period 3 over 9 points).
- DFT/FFT outputs peaks at frequencies 0, $3/9=1/3$, $6/9=2/3$ (multiples of $1/\text{period}$).
- Peak at $1/3$ implies $\text{period}=3$.
- In Shor's, QFT finds these frequencies, and post-processing (continued fractions) extracts r exactly with high probability.
- Thus, QFT turns the hard period search into efficient frequency estimation, enabling quantum factorization to break RSA-sized numbers in seconds on a large quantum computer, vs. years classically.

Sample QFT Superpositions

